API ESCOLA

Alunos, Cursos e Matrículas

Desenvolvimento de uma API REST com banco de dados SQLite e hospedagem de ponta a ponta em uma máquina virtual na Microsoft Azure

---

Linguagem: Python 3.12 + FastAPI

Banco de dados: SQLite 3

Nuvem: Microsoft Azure – VM Linux, SKU Standard_B1s

Repositório: https://github.com/RenesFernandes7366/api-alunos

Data: Outubro de 2026

Sumário

1. Introdução e objetivo.....3
1.1 Entregáveis.....3
2. Visão geral da solução.....4
2.1 Por que essa arquitetura.....4
3. Tecnologias e linguagem de programação.....6
4. Banco de dados SQLite.....7
4.1 Modelo entidade-relacionamento.....7
4.2 Regras garantidas pelo banco.....8
5. A API.....9
5.1 Estrutura do código.....9
5.2 Rotas (endpoints).....9
5.3 Códigos de resposta HTTP.....10
5.4 Caminho de uma requisição.....10
5.5 Exemplo de requisição e resposta.....10
6. Testes e evidências (prints de tela).....11
6.1 Testes automatizados (pytest).....11
6.2 Teste de ponta a ponta com chamadas HTTP.....11
6.3 Documentação interativa (Swagger UI).....12
7. Elementos de nuvem (Microsoft Azure).....16
7.1 Máquina virtual e SKU.....16
7.2 Armazenamento.....17
7.3 Rede e segurança.....17
7.4 Configuração automática da VM (cloud-init).....18
8. Passo a passo do deploy.....19
8.1 Rodar e testar localmente.....19
8.2 Publicar no GitHub.....19
8.3 Criar a infraestrutura na Azure.....19
8.4 Validar na nuvem.....19
8.5 Atualizar e remover.....20
9. Repositório e prints da nuvem.....21
9.1 Link do repositório.....21
9.2 Prints do ambiente Azure.....21
10. Melhorias futuras e conclusão.....22
10.1 Melhorias possíveis.....22
10.2 Conclusão.....22

1. Introdução e objetivo

Este documento descreve, passo a passo, o desenvolvimento e a publicação de uma **API REST básica** para o cadastro de **alunos**, **cursos** e **matrículas**. A API grava os dados em um banco **SQLite** e é hospedada de ponta a ponta em uma **máquina virtual Linux na Microsoft Azure**, ficando acessível pela internet através de um IP público.

O trabalho cobre todas as etapas do ciclo: escrita do código, testes automatizados, versionamento no GitHub, criação da infraestrutura na nuvem, configuração do servidor e validação final com chamadas HTTP reais.

1.1 Entregáveis

- **Código-fonte** da API em Python (FastAPI) com banco SQLite.
- **Repositório no GitHub** com código, testes, scripts de deploy e documentação.
- **Esquemas visuais**: arquitetura de nuvem, modelo entidade-relacionamento, fluxo de uma requisição e fluxo de deploy.
- **Prints de tela** dos testes, da documentação interativa (Swagger) e das chamadas à API.
- **Explicação dos elementos de nuvem**: VM e SKU, armazenamento, rede, segurança e sistema operacional.
- Este documento, em **Word** e **PDF**.

2. Visão geral da solução

A solução segue uma arquitetura simples de três camadas dentro de uma única máquina virtual. O usuário acessa o **IP público** da VM pela porta 80. O **Network Security Group (NSG)** permite apenas as portas 22 (SSH, administração) e 80 (HTTP). Dentro da VM, o **Nginx** recebe a requisição e a repassa para o **Uvicorn**, servidor que executa a aplicação **FastAPI**. A aplicação lê e grava no arquivo **SQLite**, que fica no disco gerenciado (Managed Disk) da VM.

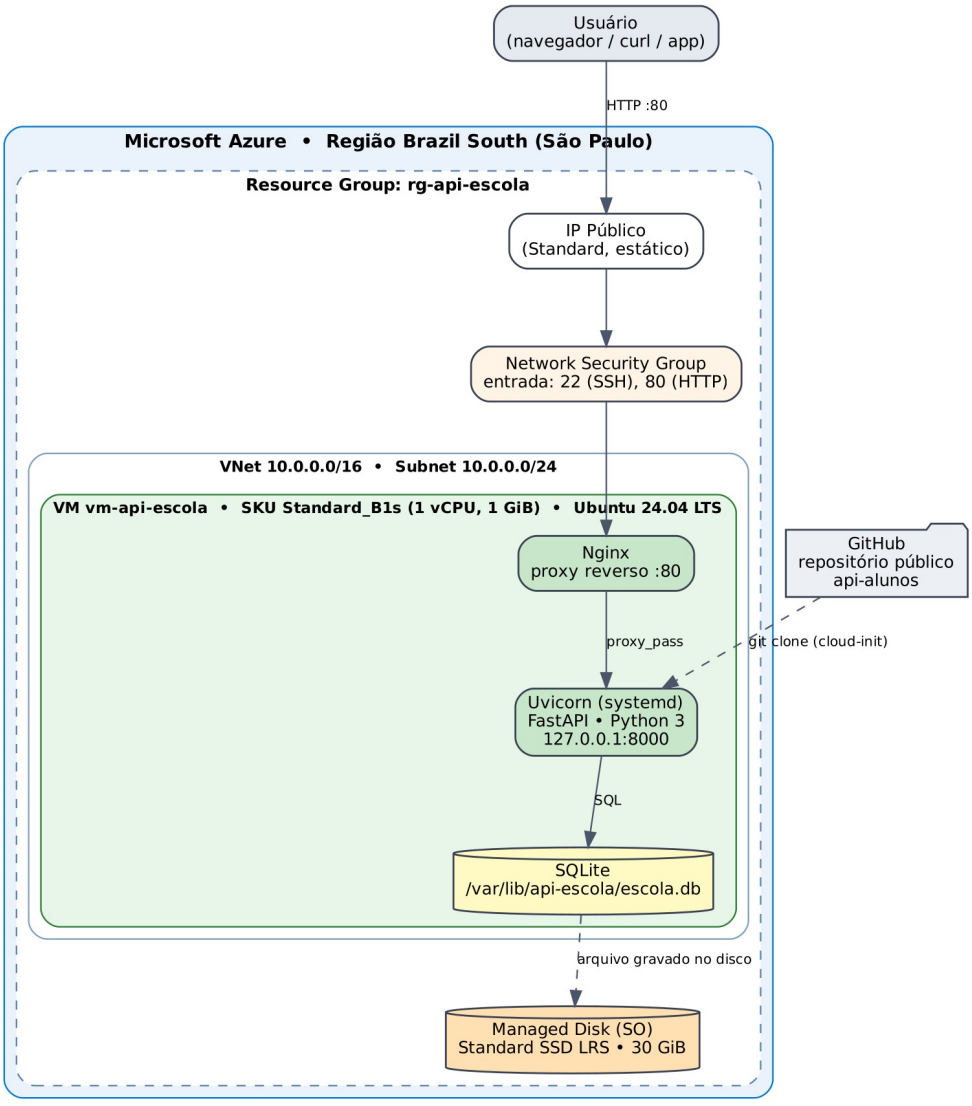


Figura 1 – *Arquitetura da solução na Azure*

2.1 Por que essa arquitetura

- **Simplicidade:** uma única VM concentra servidor web, aplicação e banco. É o modelo mais didático para entender cada peça da nuvem.
- **Custo baixo:** a SKU `Standard_B1s` é uma das menores da Azure e está entre os tamanhos incluídos nas horas gratuitas da conta free para novos clientes.

- **SQLite dispensa servidor de banco:** o banco é um arquivo no disco, sem instalação ou serviço extra. É adequado a uma API básica com uma única instância.
- **Nginx na frente:** protege o Uvicorn (que escuta só em 127.0.0.1), permite HTTPS no futuro e é o padrão de mercado para publicar aplicações Python.

3. Tecnologias e linguagem de programação

A linguagem escolhida foi **Python**, pela sintaxe clara, pelo ecossistema maduro para APIs e por já vir instalada no Ubuntu. O framework **FastAPI** foi escolhido porque valida os dados automaticamente e gera a documentação interativa (Swagger) sem código extra.

Camada	Tecnologia	Função no projeto
Linguagem	Python 3.12	Linguagem de todo o back-end (versão padrão do Ubuntu 24.04).
Framework web	FastAPI	Define as rotas HTTP, converte JSON e gera a documentação OpenAPI/Swagger.
Servidor de aplicação	Uvicorn (ASGI)	Executa a aplicação FastAPI e atende as requisições na porta 8000.
Validação	Pydantic v2	Valida tipos, tamanhos, e-mail e datas; devolve erro 422 se algo estiver inválido.
Banco de dados	SQLite 3	Banco relacional em arquivo único (`escola.db`), acessado pelo módulo `sqlite3` do Python.
Proxy reverso	Nginx	Recebe o tráfego público na porta 80 e repassa ao Uvicorn.
Gerência de serviço	systemd	Mantém a API rodando, reinicia em caso de falha e inicia no boot.
Provisionamento	Azure CLI + cloud-init	Cria os recursos na Azure e configura a VM automaticamente no primeiro boot.
Testes	pytest + httpx	8 testes automatizados cobrindo CRUD, validações e regras de negócio.
Versionamento	Git + GitHub	Histórico do código e fonte de onde a VM baixa a aplicação.

4. Banco de dados SQLite

O **SQLite** é um banco de dados relacional que guarda tudo em **um único arquivo**. Não há processo servidor: a própria aplicação abre o arquivo e executa comandos SQL. Na VM, o arquivo fica em `/var/lib/api-escola/escola.db`; o caminho é definido pela variável de ambiente `DB_PATH`.

4.1 Modelo entidade-relacionamento

O modelo tem três tabelas. Um aluno pode se matricular em vários cursos, e um curso pode ter vários alunos, um relacionamento **muitos-para-muitos** resolvido pela tabela associativa `matriculas`.

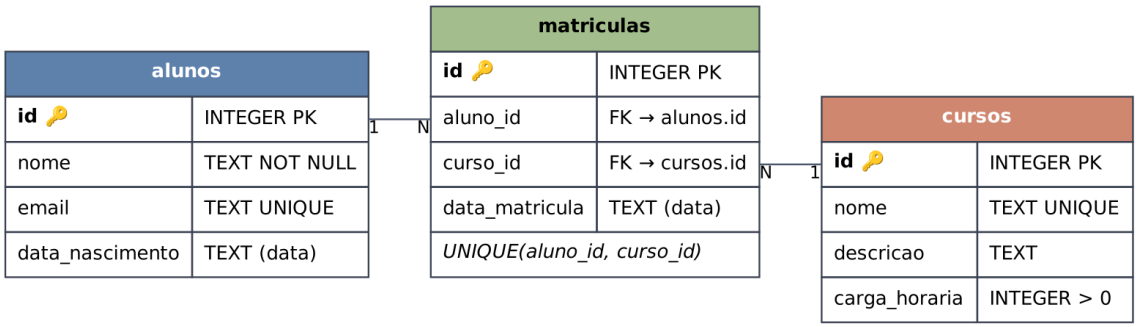


Figura 2 – *Modelo entidade-relacionamento (ER)*

Tabela	Campo	Tipo / restrição	Descrição
alunos	id	INTEGER PK AUTOINCREMENT	Identificador do aluno
alunos	nome	TEXT NOT NULL	Nome completo
alunos	email	TEXT NOT NULL UNIQUE	E-mail, sem repetição
alunos	data_nascimento	TEXT (AAAA-MM-DD)	Opcional
cursos	id	INTEGER PK AUTOINCREMENT	Identificador do curso
cursos	nome	TEXT NOT NULL UNIQUE	Nome do curso, sem repetição
cursos	descricao	TEXT	Opcional
cursos	carga_horaria	INTEGER CHECK (> 0)	Horas do curso
matriculas	id	INTEGER PK AUTOINCREMENT	Identificador da matrícula
matriculas	aluno_id	FK → alunos.id ON DELETE CASCADE	Aluno matriculado
matriculas	curso_id	FK → cursos.id ON DELETE CASCADE	Curso da matrícula

Tabela	Campo	Tipo / restrição	Descrição
matriculas	data_matricula	TEXT DEFAULT date('now')	Data automática (UTC)

4.2 Regras garantidas pelo banco

- **UNIQUE(aluno_id, curso_id):** o mesmo aluno não pode se matricular duas vezes no mesmo curso.
- **ON DELETE CASCADE:** ao remover um aluno ou curso, as matrículas dele são removidas junto, sem deixar registros órfãos.
- **PRAGMA foreign_keys = ON:** ativado em toda conexão, porque o SQLite não valida chaves estrangeiras por padrão.
- **Criação automática:** as tabelas são criadas na primeira execução (`CREATE TABLE IF NOT EXISTS`), e recriadas se o arquivo for apagado.

Trecho do esquema (app/database.py)

```
CREATE TABLE IF NOT EXISTS matriculas (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    aluno_id       INTEGER NOT NULL REFERENCES alunos(id) ON DELETE CASCADE,
    curso_id       INTEGER NOT NULL REFERENCES cursos(id) ON DELETE CASCADE,
    data_matricula TEXT    NOT NULL DEFAULT (date('now')),
    UNIQUE (aluno_id, curso_id)
);
```

5. A API

5.1 Estrutura do código

```
api-alunos/
├── app/
│   ├── main.py           # rotas da API (FastAPI)
│   ├── database.py       # conexão e criação das tabelas SQLite
│   └── schemas.py        # validação dos dados (Pydantic)
├── tests/test_api.py     # 8 testes automatizados (pytest)
├── scripts/smoke_test.sh # teste HTTP de ponta a ponta
├── deploy/
│   ├── azure_criar_vm.sh # cria a infraestrutura na Azure
│   ├── cloud-init.yaml   # configura a VM no primeiro boot
│   └── atualizar_vm.sh   # atualiza a API após novo push
├── docs/                 # diagramas e prints
├── requirements.txt      # dependências de produção
└── README.md
```

- **database.py** abre a conexão com o SQLite, ativa as chaves estrangeiras, faz commit/rollback automaticamente e fecha a conexão ao final (gerenciador de contexto `with`).
- **schemas.py** define os modelos de entrada (`AlunoIn`, `CursoIn`, `MatriculaIn`) e de saída. O Pydantic rejeita, por exemplo, e-mail inválido ou carga horária zero.
- **main.py** declara as rotas. Todas as consultas usam **parâmetros** (`?`) em vez de concatenar texto, o que impede **SQL Injection**.

5.2 Rotas (endpoints)

Método	Rota	O que faz	Respostas
GET	/health	Verifica se a API e o banco respondem	200
GET	/cursos	Lista todos os cursos	200
POST	/cursos	Cria um curso	201, 409, 422
GET	/cursos/{id}	Busca um curso	200, 404
PUT	/cursos/{id}	Atualiza um curso	200, 404, 409, 422
DELETE	/cursos/{id}	Remove um curso (e suas matrículas)	204, 404
GET	/alunos	Lista todos os alunos	200
POST	/alunos	Cria um aluno	201, 409, 422
GET	/alunos/{id}	Busca um aluno	200, 404
PUT	/alunos/{id}	Atualiza um aluno	200, 404, 409, 422
DELETE	/alunos/{id}	Remove um aluno (e suas matrículas)	204, 404

Método	Rota	O que faz	Respostas
GET	/alunos/{id}/cursos	Lista os cursos em que o aluno está matriculado	200, 404
GET	/matriculas	Lista matrículas com nome do aluno e do curso	200
POST	/matriculas	Matricula um aluno em um curso	201, 404, 409, 422
DELETE	/matriculas/{id}	Cancela uma matrícula	204, 404

5.3 Códigos de resposta HTTP

Código	Significado	Quando ocorre
200 OK	Sucesso	Consultas e atualizações bem-sucedidas
201 Created	Criado	Novo aluno, curso ou matrícula
204 No Content	Removido	Exclusão bem-sucedida
404 Not Found	Não encontrado	ID inexistente
409 Conflict	Conflito	E-mail, nome de curso ou matrícula repetidos
422 Unprocessable	Dados inválidos	JSON fora do formato (ex.: e-mail inválido)

5.4 Caminho de uma requisição

A figura abaixo mostra o que acontece quando um cliente envia `POST /matriculas`. Se o JSON for inválido, o Pydantic devolve 422 antes de tocar no banco. Se o aluno ou curso não existir, a API devolve 404; se a matrícula já existir, 409.

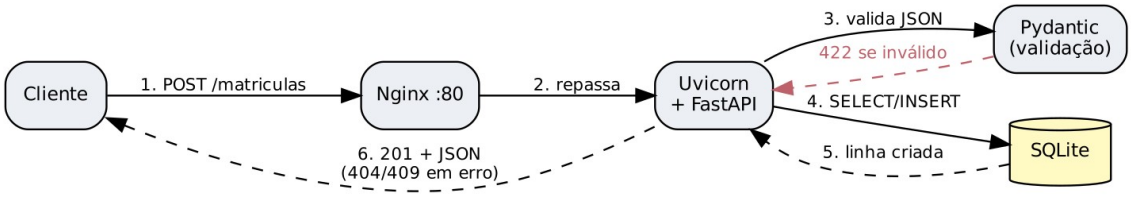


Figura 3 – *Fluxo de uma requisição dentro da VM*

5.5 Exemplo de requisição e resposta

```
curl -X POST http://IP-DA-VM/matriculas \
     -H "Content-Type: application/json" \
     -d '{"aluno_id": 1, "curso_id": 1}'

HTTP/1.1 201 Created
{"aluno_id":1,"curso_id":1,"id":1,"data_matricula":"2026-10-26",
 "aluno_nome":"Maria Silva","curso_nome":"Computação em Nuvem"}
```

6. Testes e evidências (prints de tela)

A API foi testada de duas formas: **testes automatizados** com pytest, que usam um banco SQLite temporário e isolado para cada teste, e um **teste de ponta a ponta** (`smoke_test.sh`) que faz chamadas HTTP reais com `curl` contra a API em execução. O mesmo script é usado contra o IP público da VM depois do deploy.

6.1 Testes automatizados (pytest)

Os 8 testes cobrem: health check, CRUD completo de cursos e alunos, nome de curso duplicado (409), carga horária inválida (422), e-mail inválido e duplicado, fluxo completo de matrícula (incluindo matrícula repetida e exclusão em cascata) e matrícula com IDs inexistentes (404). **Resultado: 8 de 8 aprovados.**

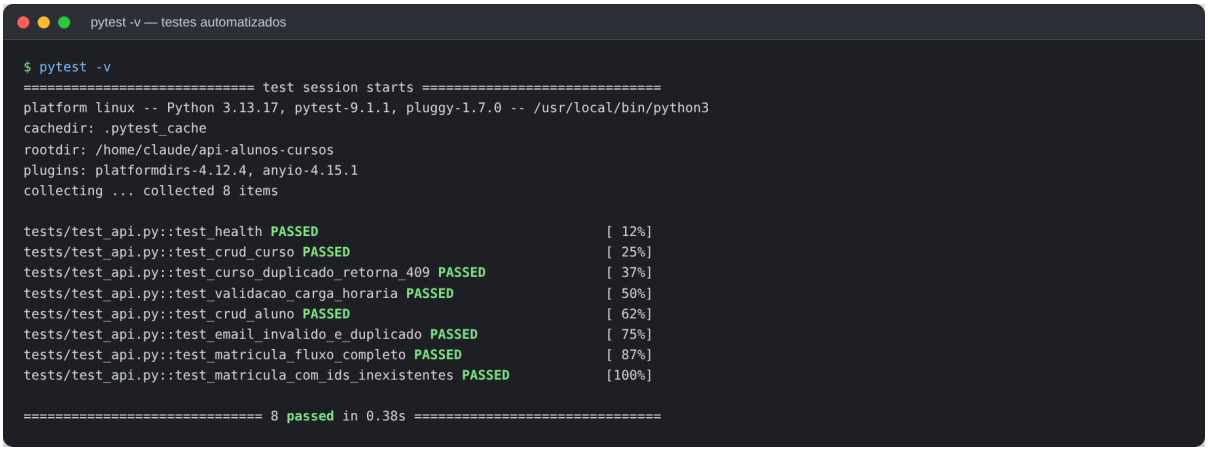


Figura 4 – *Execução dos testes automatizados – 8 testes aprovados*

6.2 Teste de ponta a ponta com chamadas HTTP

O script cria dois cursos, dois alunos e três matrículas, consulta os cursos de um aluno e confirma os erros esperados: matrícula duplicada (409) e aluno inexistente (404).

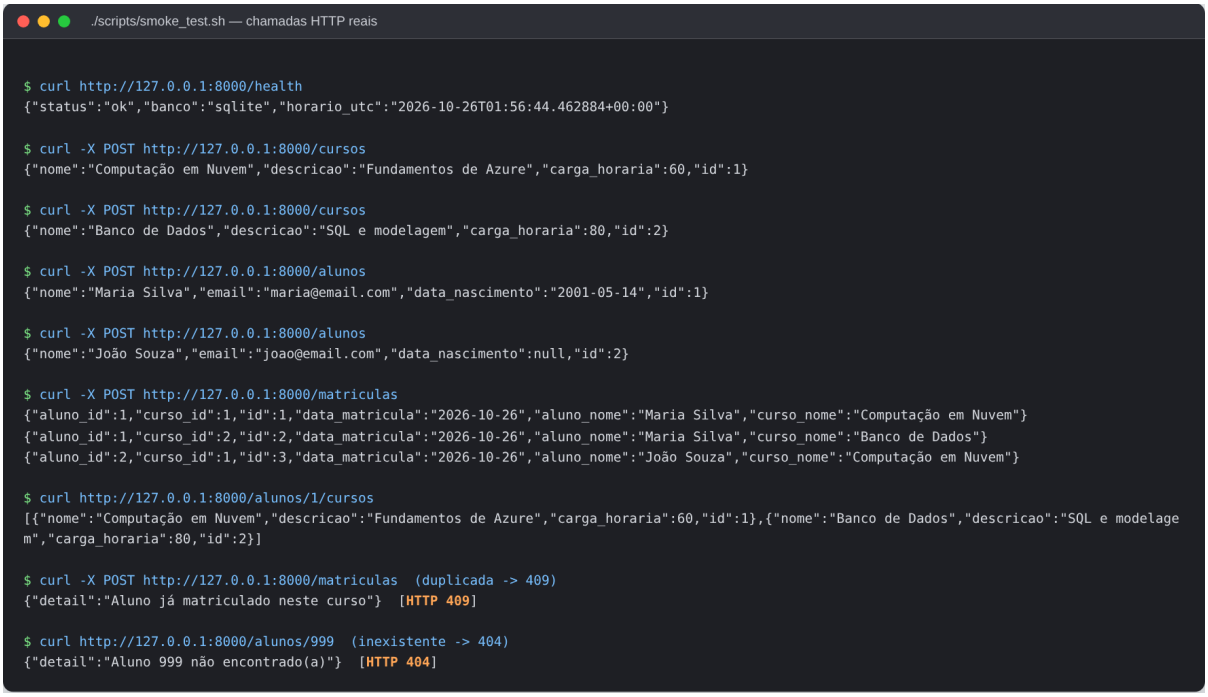


Figura 5 – *Saída do smoke_test.sh com chamadas reais à API*

6.3 Documentação interativa (Swagger UI)

O FastAPI gera automaticamente a página `/docs`, que lista todas as rotas e permite testá-las direto no navegador pelo botão **Try it out**.



Figura 6 – *Página /docs com todas as rotas agrupadas*

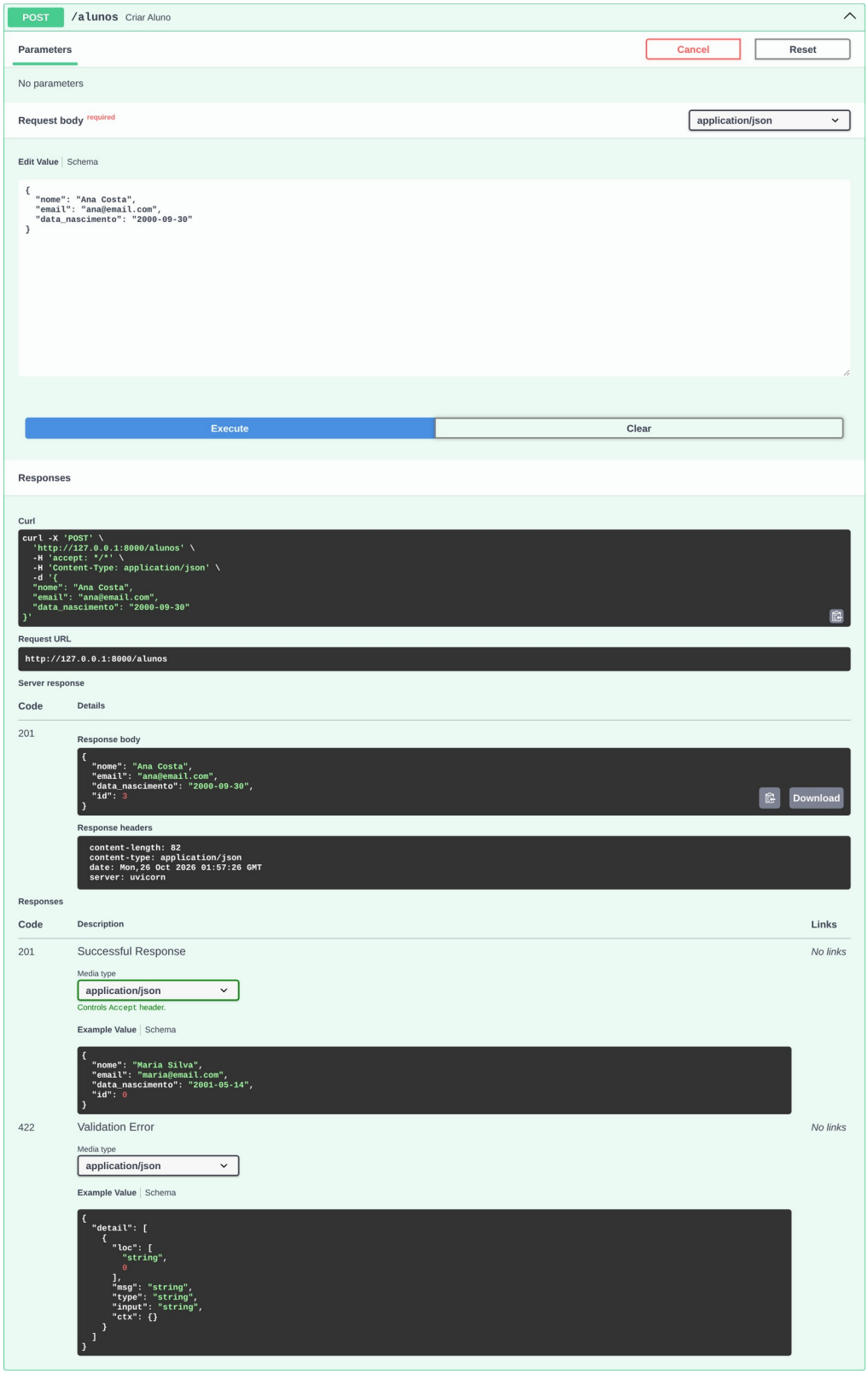


Figura 7 – *Cadastro de aluno pelo Swagger – resposta 201 Created*

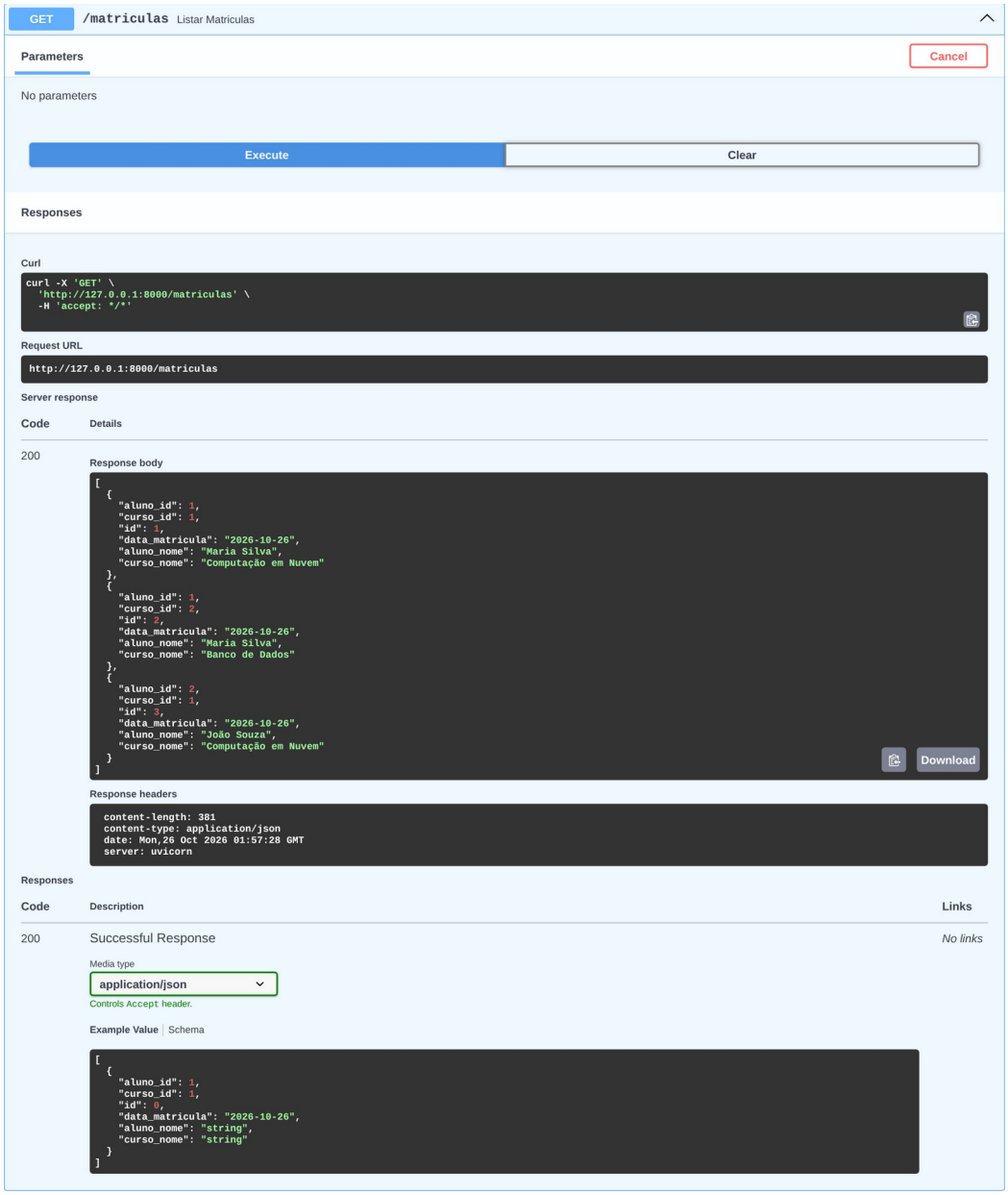


Figura 8 – *Listagem de matrículas pelo Swagger com os nomes do aluno e do curso*



Figura 9 – *Rota /health aberta no navegador*

> **Observação:** estes prints foram capturados com a API rodando localmente (endereço 127.0.0.1). Depois do deploy, as mesmas telas devem ser capturadas no endereço público `http://IP-DA-VM`, junto com os prints do portal da Azure listados na seção 9.

7. Elementos de nuvem (Microsoft Azure)

Todos os recursos são criados por um único script (`deploy/azure_criar_vm.sh`) usando a **Azure CLI**. Esta seção explica cada elemento.

Recurso	Nome / configuração	Para que serve
Resource Group	rg-api-escola	Agrupa todos os recursos do projeto; apagá-lo remove tudo de uma vez.
Região	Brazil South (São Paulo)	Datacenter mais próximo do usuário, menor latência.
Máquina virtual	vm-api-escola	Servidor Linux que executa Nginx, API e banco.
SKU (tamanho)	Standard_B1s	1 vCPU, 1 GiB de RAM, família B (burstable).
Imagem / SO	Ubuntu Server 24.04 LTS	Sistema operacional com suporte de longo prazo.
Disco do SO	Managed Disk, Standard SSD LRS, 30 GiB	Armazena o sistema, a aplicação e o arquivo SQLite.
Rede virtual	VNet 10.0.0.0/16, Subnet 10.0.0.0/24	Rede privada onde a VM fica.
Interface de rede	NIC da VM	Liga a VM à subnet e ao IP público.
IP público	SKU Standard, estático	Endereço pelo qual a API é acessada na internet.
NSG	Regras de entrada 22 e 80	Firewall: só libera SSH e HTTP.
Autenticação	Chave SSH (sem senha)	Acesso administrativo seguro à VM.

7.1 Máquina virtual e SKU

Na Azure, o **SKU** (ou *size*) define o hardware virtual da VM. O nome `Standard_B1s` se lê assim:

- **Standard:** camada de preço padrão (a camada Basic antiga foi descontinuada).
- **B:** família *burstable*. A VM acumula créditos de CPU quando está ociosa e os gasta em picos de uso. É ideal para APIs com tráfego baixo e irregular, como esta.
- **1:** uma vCPU.
- **s:** suporta discos **Premium SSD** (o projeto usa Standard SSD para economizar).

Característica	Standard_B1s
vCPUs	1
Memória RAM	1 GiB
Modelo de CPU	Burstable (créditos de CPU)
Discos de dados	até 2

Característica	Standard_B1s
Uso neste projeto	Nginx + Uvicorn + SQLite consomem bem menos de 1 GiB de RAM em repouso

A conta gratuita da Azure inclui, para novos clientes, **750 horas por mês durante 12 meses** de VMs B1s, B2pts v2 (Arm) ou B2ats v2 (AMD). Como um mês tem cerca de 730 horas, uma VM B1s ligada o mês todo cabe nesse limite. Caso a B1s não esteja disponível na região, a `Standard_B2ats_v2` é uma alternativa também coberta pela conta gratuita; basta trocar a variável `SIZE` no script.

> **Custos:** o disco, o IP público e o tráfego de saída podem ser cobrados à parte. Confirme os valores na Calculadora de Preços da Azure e apague o Resource Group ao final do trabalho com `az group delete --name rg-api-escola --yes`.

7.2 Armazenamento

O armazenamento usado é o **Managed Disk** do sistema operacional, um disco virtual gerenciado pela Azure e conectado à VM. As opções escolhidas foram:

- **Tipo Standard SSD:** latência estável e custo menor que o Premium SSD. Atende bem a um banco SQLite pequeno.
- **Redundância LRS (Locally Redundant Storage):** a Azure mantém **3 cópias** síncronas dos dados dentro do mesmo datacenter, protegendo contra falha de hardware.
- **Tamanho 30 GiB:** suficiente para o Ubuntu, Python, Nginx, a aplicação e o banco.
- **Criptografia em repouso:** todos os Managed Disks são criptografados automaticamente pela Azure (SSE com chaves gerenciadas pela plataforma).

O arquivo do banco fica em `/var/lib/api-escola/escola.db`, separado do código (`/opt/api-escola`). Assim, atualizar a aplicação com `git pull` não apaga os dados.

7.3 Rede e segurança

Prioridade	Regra NSG	Porta	Origem	Ação
1000	SSH (criada pelo az vm create)	22/TCP	Qualquer	Permitir
1010	HTTP (az vm open-port)	80/TCP	Qualquer	Permitir
65500	DenyAllInBound (padrão)	Todas	Qualquer	Negar

- O **Uvicorn escuta apenas em 127.0.0.1:8000**, então não é acessível de fora; só o Nginx fala com ele.
- O serviço roda com o usuário **www-data**, sem privilégios de administrador.
- O acesso administrativo é feito por **chave SSH**; não há senha para ser descoberta por força bruta.
- **Recomendação:** restringir a regra SSH ao IP do administrador e adicionar HTTPS (Let's Encrypt) com um nome DNS.

7.4 Configuração automática da VM (cloud-init)

O **cloud-init** é um padrão para configurar VMs Linux no primeiro boot. O arquivo `deploy/cloud-init.yaml` é enviado pela opção `--custom-data` e executa, sem intervenção manual:

1. instalação de `python3-venv`, `nginx`, `git` e `sqlite3`;
2. `git clone` do repositório do GitHub em `/opt/api-escola`;
3. criação de um ambiente virtual Python e instalação das dependências (`requirements.txt`);
4. criação do serviço **systemd** `api-escola` (inicia no boot e reinicia em caso de falha);
5. configuração do **Nginx** como proxy reverso da porta 80 para a 8000.

Serviço systemd

```
[Service]
User=www-data
WorkingDirectory=/opt/api-escola
Environment=DB_PATH=/var/lib/api-escola/escola.db
ExecStart=/opt/api-escola/.venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000
Restart=always
```

Nginx (proxy reverso)

```
server {
    listen 80 default_server;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

8. Passo a passo do deploy

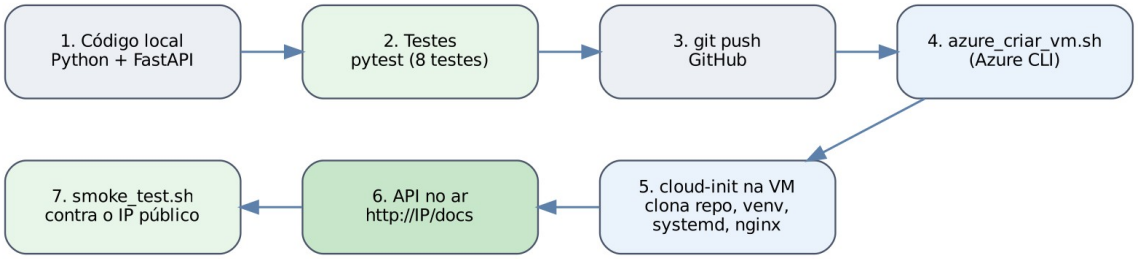


Figura 10 – *Fluxo completo: do código à API publicada*

8.1 Rodar e testar localmente

```
git clone https://github.com/RenesFernandes7366/api-alunos.git
cd api-alunos
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
pytest -v                      # 8 testes
uvicorn app.main:app --reload  # http://127.0.0.1:8000/docs
```

8.2 Publicar no GitHub

```
# crie um repositório PÚBLICO vazio chamado api-alunos no GitHub
git remote add origin https://github.com/RenesFernandes7366/api-alunos.git
git push -u origin main
```

O repositório precisa ser **público** (ou a VM precisaria de credenciais) para que o cloud-init consiga cloná-lo.

8.3 Criar a infraestrutura na Azure

```
az login
./deploy/azure_criar_vm.sh   # usa https://github.com/RenesFernandes7366/api-alunos
por padrão
```

O script cria o Resource Group, a VM com rede, IP público, NSG e disco, libera a porta 80 e mostra o IP. Após cerca de 3 minutos (tempo do cloud-init), a API responde em `http://IP-DA-VM/docs`.

No Windows, o script pode ser executado no **Azure Cloud Shell** (Bash), direto no portal, que já tem a Azure CLI e o login prontos.

8.4 Validar na nuvem

```
./scripts/smoke_test.sh http://IP-DA-VM
ssh azureuser@IP-DA-VM 'systemctl status api-escola --no-pager'
```

8.5 Atualizar e remover

```
# depois de um novo git push:
ssh azureuser@IP-DA-VM 'bash -s' < deploy/atualizar_vm.sh

# ao final do trabalho (para a cobrança):
az group delete --name rg-api-escola --yes --no-wait
```

9. Repositório e prints da nuvem

9.1 Link do repositório

Repositório: https://github.com/RenesFernandes7366/api-alunos

O repositório contém o código, os testes, os scripts de deploy, o README com instruções e a pasta `docs/` com os diagramas e prints deste documento.

9.2 Prints do ambiente Azure

Após executar o deploy, inclua aqui os prints do ambiente real:

1. Portal da Azure → **Resource Group rg-api-escola** mostrando todos os recursos criados.
2. **Visão geral da VM** (Overview) com o tamanho `Standard_B1s`, o SO e o IP público.
3. **Networking / NSG** mostrando as regras de entrada 22 e 80.
4. **Disks** mostrando o disco Standard SSD LRS de 30 GiB.
5. Navegador aberto em `http://IP-DA-VM/docs` (Swagger servido pela VM).
6. Terminal com o `smoke_test.sh` executado contra o IP público.

10. Melhorias futuras e conclusão

10.1 Melhorias possíveis

- **HTTPS** com certificado gratuito Let's Encrypt (Certbot) e um nome DNS para o IP público.
- **Backup do SQLite** com um agendamento (cron) copiando o arquivo para o **Azure Blob Storage**.
- **Autenticação** (token JWT ou chave de API) para proteger as rotas de escrita.
- **CI/CD** com GitHub Actions: rodar os testes a cada push e atualizar a VM automaticamente.
- Para muitos acessos simultâneos de escrita, migrar o banco para o **Azure Database for PostgreSQL**.

10.2 Conclusão

O projeto entrega uma API REST funcional, testada e documentada, publicada em infraestrutura de nuvem real. O código foi validado por 8 testes automatizados e por chamadas HTTP de ponta a ponta. Toda a infraestrutura é criada por script, o que torna o processo **reproduzível**: qualquer pessoa com uma conta Azure consegue recriar o ambiente em poucos minutos com um único comando, e removê-lo da mesma forma.